

1. テストケースを明確にしおくべき → TODOとして整理

1. 小さいテストを一つ書く
2. おバ2のテストを実行し、失敗があることを確認する
3. 小さい変更を行う
4. 再びテストを実行し、成功することを確認する
5. リファクタを行い、重複を除去する

仮実装として、土台をつくる
TDDは=4に肉づいて2に
いくことで、"良い"コードにする

⇒ おおまかな実装により、新しいTODOが生まれる。

2. TDDの基本サイクル

この4段階で、実装方法
が見えてくることもある

1つを実装して42にすることで、
テスト項目として何が必要か見えてくる。

1. テストを書く
2. テストを動かす
3. 正しい実装に変える

Point) 両値観をテストに翻訳する

難しいものは、仮実装(章)から徐々にいく。

実装する際、42が244か、
どのように呼ぶか決まらねば。
書いてしまえば、中身は作る
というものの時

3. 三角測量の考え方を導入 (各前だけ一丁前にしている気がするが)

→ 実装を20以上、テストコードに42、一つの機能の動作チェック

4. 実装された機能を使用して、テストコードの品質向上を図る

⇒ より結合度の低い実装テストをすることを目指す

5. 基本的な実装の流れは、第1章の5ステップの通り

⇒ 1~4まじりは重複のある非スマートなコードでも良い!

← 抽象化もどの段階で行うのか
という問題なだけ。個人的には
好きな考え方はないかも。

6. とにかく"し"の変更 → テストを動かす という流れを軸に行うことは開発効率↑につながる
必要があれば、テスト不足も解消する (機能追加などがある場合、あたり前 (= テストを増える))

7. 何か疑念があれば、テストに翻訳し残しておく

8. テストコード内のネストを、コード内でサブリナスとして定義すること2"隠す" (≒ 15%ルール)

9. 共通部分は、なるべく上の階層に持ち上げる

随時、歩幅 (≒ 一度の実装量) を調整するべき

10. テスト側の論理構造に不整合があれば、"適切なタイミング"で修正する。

11. 1つずつ"コリコリ"実装を進める ⇒ コードをまとめ、不要なコードを消していく
→ この流れで、より簡潔で、精度の良いテストを作成する

12. 実装の抽象化は、具体性の高いものからテストを通して2"く"のように進め2"く"と良い

- ・ テストや再利用がしやすいか
- ・ 責務の所在は適切か

13. -

14. (学習用テスト ... その機能がどのような動作をするのか確認するためのテスト
(回归テスト ... 不具合を再現するようなテスト
⇒実装には、回归テストを上手に管理する必要アリ

15. 変更は { テスト任せ ... テストコードの不具合を検知
コンパイラ任せ ... コンパイルエラー } ケースによって使い分ける

16. フライスキャを用いて、テストコードの最適化を行う必要アリ

17. リファクタリング { 不要な重複は無い
テストは足りている
例) こう動くように
コードを批判的に見る必要がある

テスト品質の評価方法として、以下の2つがある

{ スタートメントカバレッジ
バグ挿入
詳細は後述

18~24 テストを実行する環境の構築方法 → 現状、興味がないのでスルー

25. テストの実装 + 実行は、開発ストレスの低減や効率の上昇が見込める
・テスト同士は独立させるべき → 個別テスト(小さく2速いテスト)を行うことが出来る
・テストケースや実装する機能はTODOリストに書き出す → 漏れをなくす
・テストを書く時の効果的な思想
{ ・テストファースト 後で書くのはストレスになる
・アサートファースト アサートの結果が得られやすくなる
テストデータには、より多くの情報を含ませ、本物に近く、敢えて明示的に書くことが出来る

26. テスト実装手順は レッド → グリーン → リファクタリング が効率的
簡単に書けそう、実装することによって学びがありそうなものから書く
・脱線するものはなく、TODOリストに残す
・回归テスト(エラーを再現するテスト)をしっかりと書く

27. 一度に大きいテストを書くべきではない → "小さく書く2テスト"を繰り返すのが、速く効率的
この章はあ利便性が高いかな、たー

28. 仮実装 → 三角測量や明白な実装に直す
テストは動かす 抽象化の 必要は理解する 必要は理解する
実装は後で書く 必要は理解する 必要は理解する

29. テストが想定通り動作しているかをチェックする アサーションメッセージとあるに頼り
- ・アサーション 具体的な条件式をもって、真偽値を判断するのが安定
 - ・フィクスチャ テストの準備と後処理のことで、共通部分は抽象し、
それぞれのメソッド内部で、状態を閉じこめる（初期化と解放）必要がある
- テストメソッド名には、最大限意味を含め、テストコードが長くなり分割し、アウトラインを書く
try-catch 構文を用いて、例外処理のテストも行うことができる

30. TDDにおける設計パターン（デザインパターン）
- ・ Command 処理の実行をオブジェクトに専用に定義して行う
 - ・ Value Object Object を Value Object として扱う設計で、オブジェクト操作はコピーを渡す
これにより、別名参照問題を防ぐことができる
 - ・ Null Object Null チェックなどの条件分岐系構文を特定のメソッドに格納し、プログラバブルな
 - ・ Template Method 処理順序を抽象メソッドを用いて表現する
 - ・ Pluggable Object 条件分岐をまとめる（上記と似ている）
 - ・ Pluggable Selector 余計なサブクラスを作らずに済ます
 - ・ Factory Method メソッドを用いて、間接層を設ける
 - ・ Imposter
 - ・ Composite
 - ・ Collecting Parameter
 - ・ Singleton
-) *
- ~

31. この本で紹介されている11の7つ方法に興味はねえです。

32. テストをどこまで書くのかの目安として、「不安が退屈に変わるまで書く」
具体的には、条件分岐やループ、操作、データフローなど
- ・ テストケースをどこくらい作るかについては、そのコードの重要性や複雑性による